Contents

Milestone 5: New Feature Implementation 2

Roles in the UI, and Clean Architecture Refactor 2

- Overview 2
- Goals 2
- Requirements 2
  - 1. Updated Design Document 2
  - 2. Feature Implementation in the Next.js UI 2
  - 3. Role Application in UI and API 2
  - 4. Clean Architecture Refactor: Repositories and Services 3
- Deliverables 3
- Rubric (100%) 3

Milestone 5: New Feature Implementation

Roles in the UI, and Clean Architecture Refactor

Overview

By this point in the project students have completed:

- Designed a new feature (Milestone 1 and Milestone 2).
- Ported the UI to Next.js and deployed it to Vercel.
- Built the API for the feature in Milestone 3.
- Added OAuth-based authentication with admin and user roles.

Milestone 5 completes the full vertical slice by implementing the feature in the UI, enforcing user roles, and refactoring the API using Clean Architecture principles.

Goals

- Implement a role-aware feature end-to-end.
- Apply Clean Architecture: thin routes, services, repositories.
- Enforce RBAC (role-based access control) in UI and API.

Requirements

1. Updated Design Document

The updated design document must include:

- Feature summary reflecting the final implementation.
- Revised user stories (include admin and user variants).
- Updated data model and ER diagram.
- Final API design with endpoints and role rules.
- Final UI/UX flow for the feature.
- Security and RBAC description.
- Architecture overview showing UI → API → Services → Repositories.
- Assumptions, constraints, references, and AI usage.

2. Feature Implementation in the Next.js UI

- Implement all user and admin use cases for the feature.
- Integrate the feature into the app navigation.
- Use Next.js conventions for server/client components.
- Call your API (Milestone 3 + feature endpoints).
- Provide responsive UI for success and error handling.
- Implement guest behavior (redirect or limited view).

3. Role Application in UI and API

- UI must differentiate guest vs user vs admin.
- Use NextAuth session roles in UI components.
- API routes must enforce RBAC (401/403/404 where appropriate).
- UI must hide actions users cannot perform; API must still enforce.

4. Clean Architecture Refactor: Repositories and Services

- Introduce repository layer: move SQL out of routes and into modules under src/lib/repositories.
- Introduce service layer: business logic and orchestration under src/lib/services.
- API routes become controllers that call services and return HTTP responses.
- Repositories handle all SQL; services handle rules; routes handle HTTP.

Deliverables

- Updated design document (2–3 pages).
- GitHub repository URL and Vercel deployment URL.
- Captioned screenshots
    - Screenshots of role differences (code): Guest, User, Admin.
    - Screenshots of role differences (running UI): Guest, User, Admin.
- Architecture evidence showing repositories and services.
    - Screenshots of updated source tree.
- Short reflection on Clean Architecture.

Rubric (100%)

- Feature completeness (30%).
- RBAC correctness (20%).
- Clean Architecture implementation (25%).
- Clean Architecture reflection (5%)
- Design document quality (10%).
- References and professionalism (10%).